

Violence Detection

Project Presentation

Prepared by:
Sadman Sakib Khan Promit

Institution:
North South University

Course:
Pattern Recognition

Semester:
January 2023

About

The surveillance system is the mechanism for implementing security by watching over and eavesdropping masses. Modern designs comprise closed-circuit televisions (CCTV) feeding footage to security guards at the endpoint. This technique is very costly and erroneous. I have devised our project to mitigate the processes involved by eliminating the use of human labor. This way, I can avoid many human errors, such as false alerts, and most importantly, minimize costs. I can achieve this milestone by using Deep Learning. With the help of computers, this system will be able to watch over and eavesdrop on masses on its own, without the need for any human being. I used **ResNet152** for spatial and **LSTM** for sequential feature extraction for the video feed classification tasks. Thus, the system will be able to detect any anomalies threatening the safety and security of the public without involving any human being, avoiding privacy compromise.

Dataset Statistics

Real Life Violence Situations Dataset [[Kaggle Link](#)]

Size: 1.81 GB [unzipped]

Number of Classes: 2

Number of Samples: 2000

Violence samples: 1000

NonViolence samples: 1000



Initial Discussion

Motivation

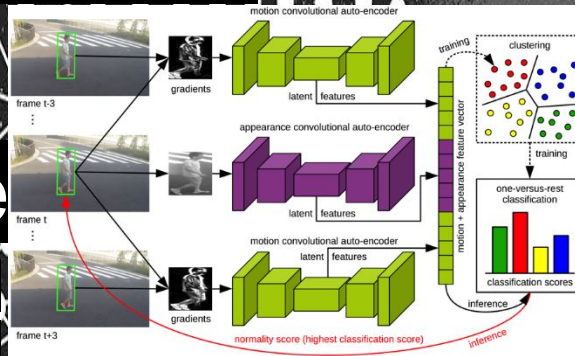
The system will be able to detect the following events

~~Face Detection~~

~~Fire~~

~~Violence~~

~~Weapon~~



Convolution

Pooling

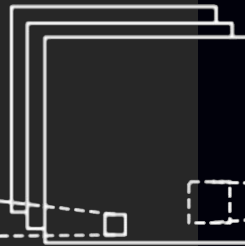
Convolution

Pooling

Fully Connected

Fully Connected

Output Prediction



Algorithms

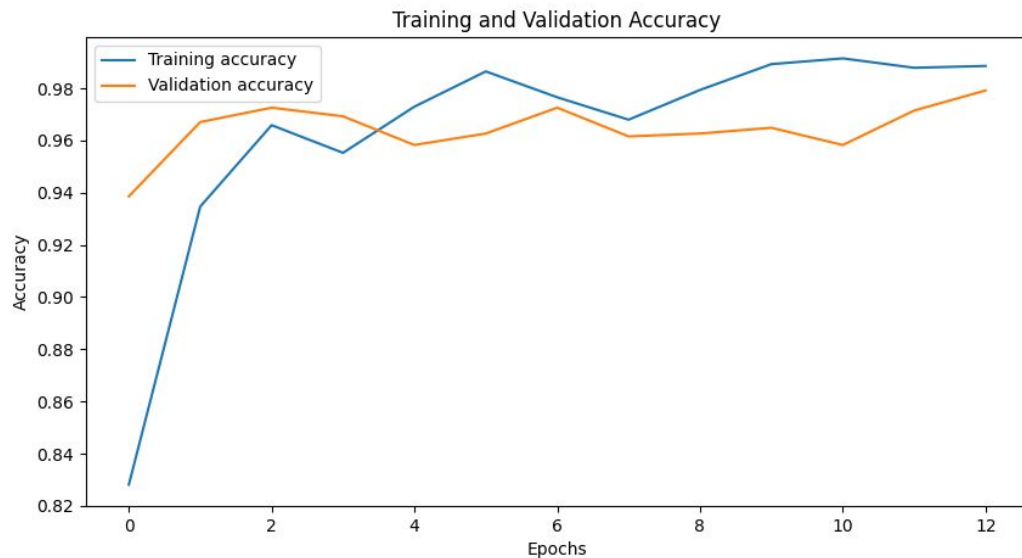
- CNN
- LSTM



Related Works

1. N. Bagga, G. Singh, B. Balusamy and A. Shanker Singh, "Violence Detection in Real Life Videos using Convolutional Neural Network," 2022 2nd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE), Greater Noida, India, 2022, pp. 872-876
2. Mumtaz, N.; Ejaz, N.; Aladhadh, S.; Habib, S.; Lee, M.Y. Deep Multi-Scale Features Fusion for Effective Violence Detection and Control Charts Visualization. *Sensors* 2022, 22, 9383.
3. Halder, R., Chatterjee, R. CNN-BiLSTM Model for Violence Detection in Smart Surveillance. *SN COMPUT. SCI.* 1, 201 (2020).

Training Curve



Stats

Total Training Time: 5:25:11 & Average Epoch Time: 0:25:00.846154

2023-06-08 15:40:09,634 - __main__ - INFO - No pre-trained model found

2023-06-08 16:04:51,578 - __main__ - INFO - Saved model checkpoint to
'behavior_analysis.pt'

2023-06-08 16:04:51,578 - __main__ - INFO - Epoch: 01 | Time: 24m 42s

2023-06-08 16:04:51,578 - __main__ - INFO - Train Loss: 0.369 | Train Acc: 82.81%

2023-06-08 16:04:51,578 - __main__ - INFO - Val. Loss: 0.186 | Val. Acc: 93.86%

2023-06-08 16:29:41,016 - __main__ - INFO - Saved model checkpoint to
'behavior_analysis.pt'

2023-06-08 16:29:41,016 - __main__ - INFO - Epoch: 02 | Time: 24m 49s

2023-06-08 16:29:41,017 - __main__ - INFO - Train Loss: 0.175 | Train Acc: 93.47%

2023-06-08 16:29:41,017 - __main__ - INFO - Val. Loss: 0.106 | Val. Acc: 96.71%

2023-06-08 16:54:46,321 - __main__ - INFO - Saved model checkpoint to
'behavior_analysis.pt'

2023-06-08 16:54:46,321 - __main__ - INFO - Epoch: 03 | Time: 25m 5s

2023-06-08 16:54:46,321 - __main__ - INFO - Train Loss: 0.083 | Train Acc: 96.59%

2023-06-08 16:54:46,321 - __main__ - INFO - Val. Loss: 0.095 | Val. Acc: 97.26%

2023-06-08 17:19:15,629 - __main__ - INFO - Saved model checkpoint to
'behavior_analysis.pt'

2023-06-08 17:19:15,629 - __main__ - INFO - Epoch: 04 | Time: 24m 29s

2023-06-08 17:19:15,629 - __main__ - INFO - Train Loss: 0.108 | Train Acc: 95.53%

2023-06-08 17:19:15,629 - __main__ - INFO - Val. Loss: 0.075 | Val. Acc: 96.93%

2023-06-08 17:43:41,511 - __main__ - INFO - Epoch: 05 | Time: 24m 26s

2023-06-08 17:43:41,511 - __main__ - INFO - Train Loss: 0.077 | Train Acc: 97.30%

2023-06-08 17:43:41,511 - __main__ - INFO - Val. Loss: 0.087 | Val. Acc: 95.83%

2023-06-08 18:08:21,208 - __main__ - INFO - Epoch: 06 | Time: 24m 40s

2023-06-08 18:08:21,209 - __main__ - INFO - Train Loss: 0.031 | Train Acc: 98.65%

2023-06-08 18:08:21,209 - __main__ - INFO - Val. Loss: 0.111 | Val. Acc: 96.27%

2023-06-08 18:33:08,957 - __main__ - INFO - Epoch: 07 | Time: 24m 48s

2023-06-08 18:33:08,957 - __main__ - INFO - Train Loss: 0.058 | Train Acc: 97.66%

2023-06-08 18:33:08,957 - __main__ - INFO - Val. Loss: 0.080 | Val. Acc: 97.26%

2023-06-08 18:57:59,653 - __main__ - INFO - Epoch: 08 | Time: 24m 51s

2023-06-08 18:57:59,653 - __main__ - INFO - Train Loss: 0.077 | Train Acc: 96.80%

2023-06-08 18:57:59,653 - __main__ - INFO - Val. Loss: 0.096 | Val. Acc: 96.16%

2023-06-08 19:24:33,831 - __main__ - INFO - Epoch: 09 | Time: 26m 34s

2023-06-08 19:24:33,832 - __main__ - INFO - Train Loss: 0.038 | Train Acc: 97.94%

2023-06-08 19:24:33,832 - __main__ - INFO - Val. Loss: 0.097 | Val. Acc: 96.27%

2023-06-08 19:50:25,616 - __main__ - INFO - Epoch: 10 | Time: 25m 52s

2023-06-08 19:50:25,616 - __main__ - INFO - Train Loss: 0.029 | Train Acc: 98.93%

2023-06-08 19:50:25,617 - __main__ - INFO - Val. Loss: 0.099 | Val. Acc: 96.49%

2023-06-08 20:15:26,345 - __main__ - INFO - Epoch: 11 | Time: 25m 1s

2023-06-08 20:15:26,345 - __main__ - INFO - Train Loss: 0.024 | Train Acc: 99.15%

2023-06-08 20:15:26,345 - __main__ - INFO - Val. Loss: 0.099 | Val. Acc: 95.83%

2023-06-08 20:40:27,453 - __main__ - INFO - Epoch: 12 | Time: 25m 1s

2023-06-08 20:40:27,453 - __main__ - INFO - Train Loss: 0.032 | Train Acc: 98.79%

2023-06-08 20:40:27,453 - __main__ - INFO - Val. Loss: 0.093 | Val. Acc: 97.15%

2023-06-08 21:05:20,740 - __main__ - INFO - Epoch: 13 | Time: 24m 53s

2023-06-08 21:05:20,740 - __main__ - INFO - Train Loss: 0.033 | Train Acc: 98.86%

2023-06-08 21:05:20,740 - __main__ - INFO - Val. Loss: 0.098 | Val. Acc: 97.92%

2023-06-08 21:30:17,674 - __main__ - INFO - No improvement in validation loss for 10
epochs. Stopping training.

Regularization & Optimization

Optimization: The optimizer used in this script is **Adam**. The learning rate scheduler used here is **ReduceLROnPlateau** from PyTorch. This scheduler dynamically adjusts the learning rate based on some validation measurements. If the validation loss does not improve for a predefined number of epochs (here, defined by the 'patience' parameter as 10), the learning rate is reduced by a factor (here set to 0.1). This helps in achieving better generalization performance by effectively shrinking the learning rate, thereby slowing down learning when model performance stops improving on the validation set. The script also uses **gradient accumulation** to handle the cases where the batch size can't be increased due to memory limitations. Additionally, the code implements **mixed-precision** training using `torch.cuda.amp.GradScaler`. It enables the model to use both 16-bit and 32-bit floating-point types during training to make it run faster and use less memory.

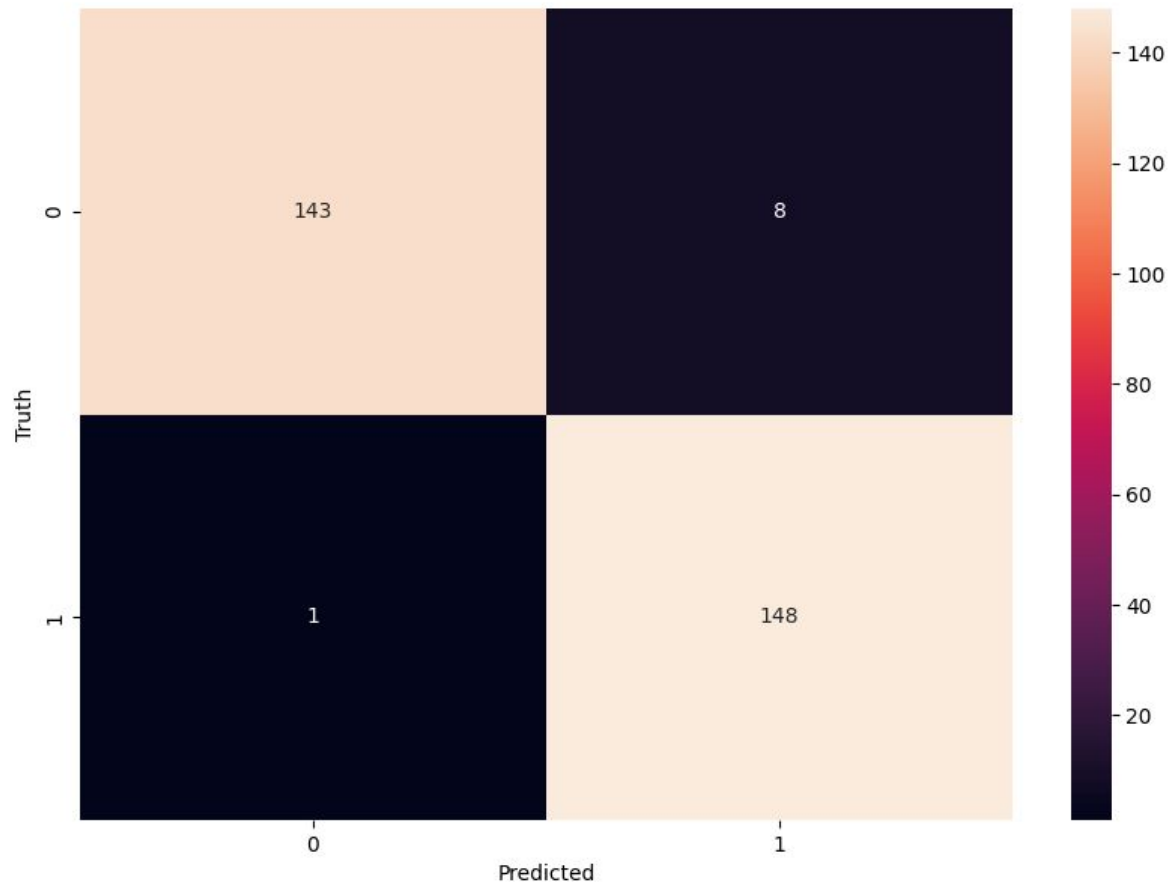
Regularization: **Dropout** is used as a regularization method. **Data augmentation** techniques (such as resizing, normalizing) are also used, which also help in regularizing the model, making it more robust and helping it generalize better. The use of **early stopping** also serves as a form of regularization. Training is stopped after the validation loss stops improving after a certain number of epochs, preventing the model from overfitting to the training data. The model uses a pre-trained ResNet-152 model as a feature extractor, which can also be seen as a form of transfer learning regularization. Since these weights have been pre-trained on a large dataset, they are already regularized. **Gradient checkpointing** is used to reduce memory consumption during the backward pass in the model's forward method.

Test Metrics

Accuracy: 0.97

Precision: 0.9710345441595443

Recall: 0.97



Test Metrics on other Datasets

I've combined three different datasets for testing, taking all the violence samples available and taking the same amount of NonViolence samples randomly: Anomaly-Detection-Dataset-UCF [[Kaggle Link](#)] [Positive samples], UCF101 Videos [[Kaggle Link](#)] [Negative samples] & TikTok Trending Videos [[Kaggle Link](#)] [Negative samples].

Size: 3.14 GB [unzipped]

Violence samples: 103

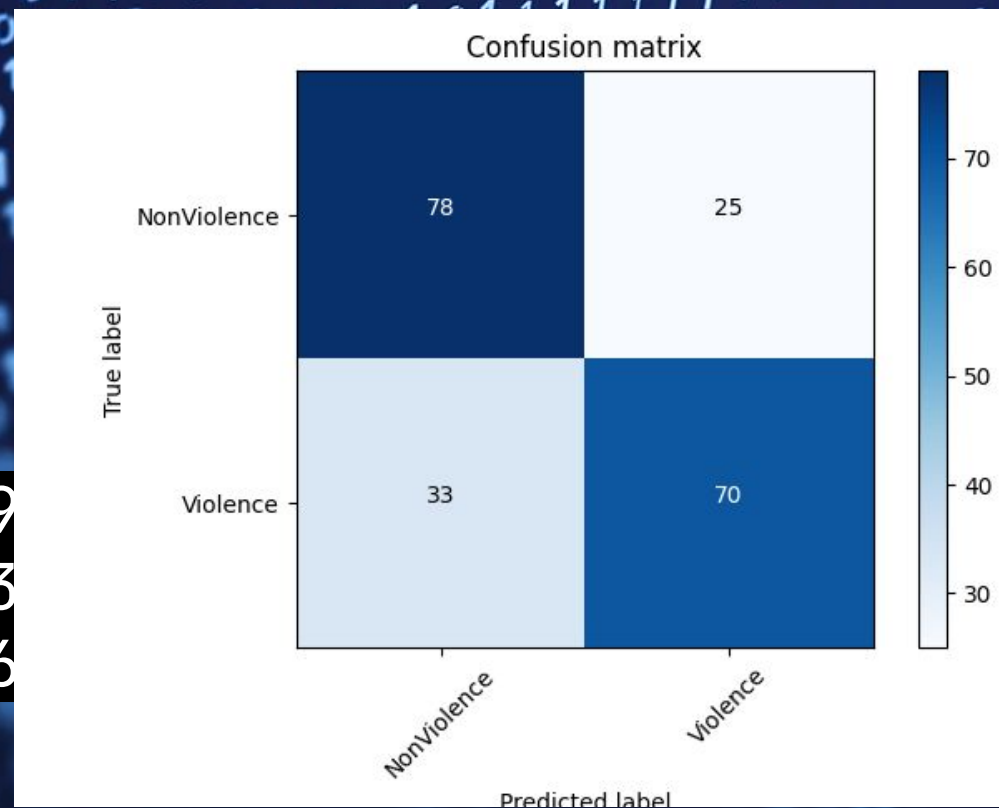
NonViolence samples: 103

Metrics:

Accuracy: 0.7184466019

Precision: 0.719772403

Recall: 0.718446



Conclusion

- Train on all the violence datasets available online.
- Combine with Natural Language Processing (NLP) to extract audio language and analyze sentiment.
- By training on hate speech datasets, I can detect violence precisely.
- Implement these models on hardware such as drone-attached cameras and Wi-Fi to practically surveil real life.
- I will try to work on the research paper if all works well.

The background features a dark blue gradient with a glowing blue wireframe mesh that resembles a molecular or network structure. Scattered throughout are numerous out-of-focus, colorful bokeh lights in shades of yellow, orange, red, and light blue.

Thank You